

Ministry of Education and Research of the Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory Work #9

**Automatic Control in Embedded Systems. Part 1:
ON-OFF Control with Hysteresis**

Author:

Alexandru Cebotari

Group FAF-233

Verified by:

Alexei Martiniuc

Technical task

Purpose

This report documents only Part 1 of the laboratory condition. The developed application implements ON-OFF automatic control with hysteresis for a measurable physical parameter using Arduino Uno, a 10k NTC thermistor with a resistive divider, a configurable setpoint, a binary output for a heating resistor and indicator LED, LCD feedback, STDIO reporting, and Arduino Serial Plotter output.

Objectives

- implement a modular ON-OFF controller with hysteresis for temperature regulation;
- acquire the temperature-related electrical signal from a thermistor divider and convert it into a physical value;
- provide configurable setpoint, hysteresis width, and acceptable minimum and maximum temperature limits;
- actuate a binary heating output and expose the actuator state on LCD and serial STDIO;
- publish `SetPoint`, `Value`, and `Output` fields for Arduino Serial Plotter;
- keep the project structured in reusable modules and validate it through hardware tests.

Individual task and interpretation

The implementation covers only Part 1 of Lab 9. The final setup uses a thermistor-resistor divider as a practical temperature sensing solution, a potentiometer for local setpoint adjustment, STDIO commands for advanced runtime control, and a binary heating/output proxy on Arduino Uno. The execution model uses sequential periodic tasks with justified recurrences, which is explicitly allowed by the laboratory condition.

1 1. Domain analysis

1.1 1.1 Motivation

The purpose of automatic control is to keep a measurable physical quantity close to a desired reference value. In embedded systems, this is achieved by repeatedly acquiring the process value, comparing it to the setpoint, and deciding how the actuator should respond. For simple temperature control, ON-OFF regulation with hysteresis remains a

practical solution because it is easy to implement, easy to validate, and sufficiently robust for low-cost embedded applications.

The main issue of plain threshold switching is rapid toggling near the reference value. Hysteresis solves this by defining a lower threshold and an upper threshold around the setpoint. The actuator turns on below the lower threshold, turns off above the upper threshold, and keeps its previous state in between.

1.2 1.2 Hardware and software context

- Hardware platform: Arduino Uno.
- Sensor subsystem: 10k NTC thermistor, 10k fixed resistor, analog acquisition on A0.
- Setpoint subsystem: potentiometer on A1 and serial command interface.
- Actuation subsystem: heating resistor and indicator LED driven from D8.
- Display subsystem: LCD1602 in 4-bit parallel mode.
- Development resources: PlatformIO, Arduino framework, LaTeX, PlantUML.
- Monitoring interfaces: serial monitor and Arduino Serial Plotter.

The hardware chosen for the final project reflects the practical constraints of the available components. Instead of using a digital temperature sensor and a relay module, the implementation uses a thermistor divider and a resistive heating/output proxy. This still demonstrates the full control loop: measurement, setpoint selection, hysteresis-based decision, binary output actuation, and runtime reporting.

1.3 1.3 Existing solutions and references

Temperature control examples for Arduino often use ON-OFF logic with simple threshold comparison. Typical educational projects rely on digital sensors such as DHT22 or analog sensors such as thermistors. The most useful ideas extracted from these references are: modular driver separation, clear signal conversion steps, and runtime visibility through LCD and serial monitoring. Unlike basic examples, the current project also introduces configurable hysteresis, acceptable operating range limits, and structured reporting for validation.

1.4 1.4 Problem definition

The problem is to design and implement a modular embedded application that reads a temperature-related process variable, compares it to a configurable setpoint, applies ON-OFF control with hysteresis, drives a binary heating/output signal, and reports the relevant values through LCD and serial interfaces. The solution must remain within the Part 1 scope and exclude PID control.

Section conclusion

The domain analysis justifies the use of ON-OFF control with hysteresis, identifies the hardware-software context, and frames the final application as a complete Part 1 automatic-control solution suitable for real hardware validation.

2 2. Conceptualization and design

2.1 2.1 Design motivation

The design stage is necessary to transform the laboratory condition into a verifiable embedded application. It defines technical requirements, system architecture, behavioral diagrams, and validation criteria. This approach follows the documentation guide and ensures traceability between requirements, implementation, and test evidence.

2.2 2.2 Technical requirements

ID	Requirement	Type	Acceptance target
R1	The application shall acquire the thermistor divider value periodically and convert it to temperature.	F	measured value is refreshed every cycle
R2	The application shall support configurable setpoint selection from potentiometer and serial commands.	F	setpoint changes during runtime
R3	The application shall implement ON-OFF control with configurable hysteresis.	F	output changes according to the hysteresis band
R4	The application shall drive one binary actuator output according to the controller result.	F	output state follows control logic
R5	The application shall display measured value, setpoint, and output state on LCD.	F	LCD remains readable and updated
R6	The application shall report runtime values through serial STDIO.	F	serial log is structured and periodic
R7	The application shall send SetPoint , Value , and Output to Arduino Serial Plotter.	F	plotter-compatible line is generated
R8	The application shall allow runtime configuration of acceptable minimum and maximum temperature limits.	F	min/max thresholds are editable
R9	The software shall be organized into reusable modules for acquisition, control, actuation, and reporting.	NF	source tree follows layered separation
R10	The execution model shall use sequential tasks with justified recurrences.	NF	periodic scheduling is explicit
R11	The project shall build in PlatformIO for Arduino Uno.	NF	successful build is obtained
R12	The application shall avoid rapid toggling near the setpoint.	NF	hysteresis prevents chatter
R13	The application shall keep the Part 1 scope only and exclude PID logic.	NF	no Part 2 functionality is implemented

Table 1: Technical requirements for Lab 9 Part 1

2.3 2.3 System components and roles

Component	Role in the application
NTC thermistor and fixed resistor	Generate the analog divider voltage used for temperature estimation.
Setpoint potentiometer	Provides a local hardware method for selecting the desired temperature.
Serial command interface	Allows changing setpoint, hysteresis, acceptable limits, and reporting mode during runtime.
ON-OFF controller with hysteresis	Compares measured temperature against the reference and hysteresis band to decide the output state.
Binary heating/output proxy	Represents the actuator controlled by the software.
LCD1602	Displays measured temperature, setpoint, mode, and output state.
STDIO reporting	Provides detailed runtime information and Serial Plotter fields.
Sequential scheduler	Executes acquisition, processing, actuation, and reporting tasks with justified recurrences.

2.4 2.4 Structural architectural design

The software architecture is divided into four logical layers:

- **Application layer:** scheduler and shared application state.
- **Task layer:** acquisition, setpoint management, control, actuation, and reporting tasks.
- **Service layer:** thermistor conversion, hysteresis control, and serial command interpretation.
- **Driver layer:** thermistor input, potentiometer input, output control, LCD, and serial interfaces.

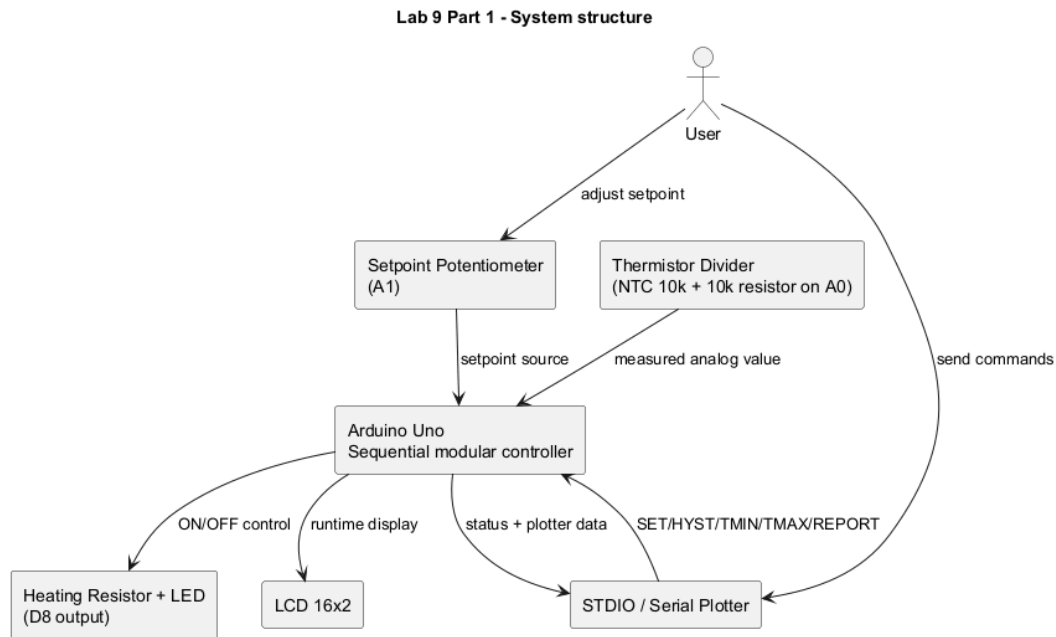


Figure 1: System structure for the thermistor-based ON-OFF control application

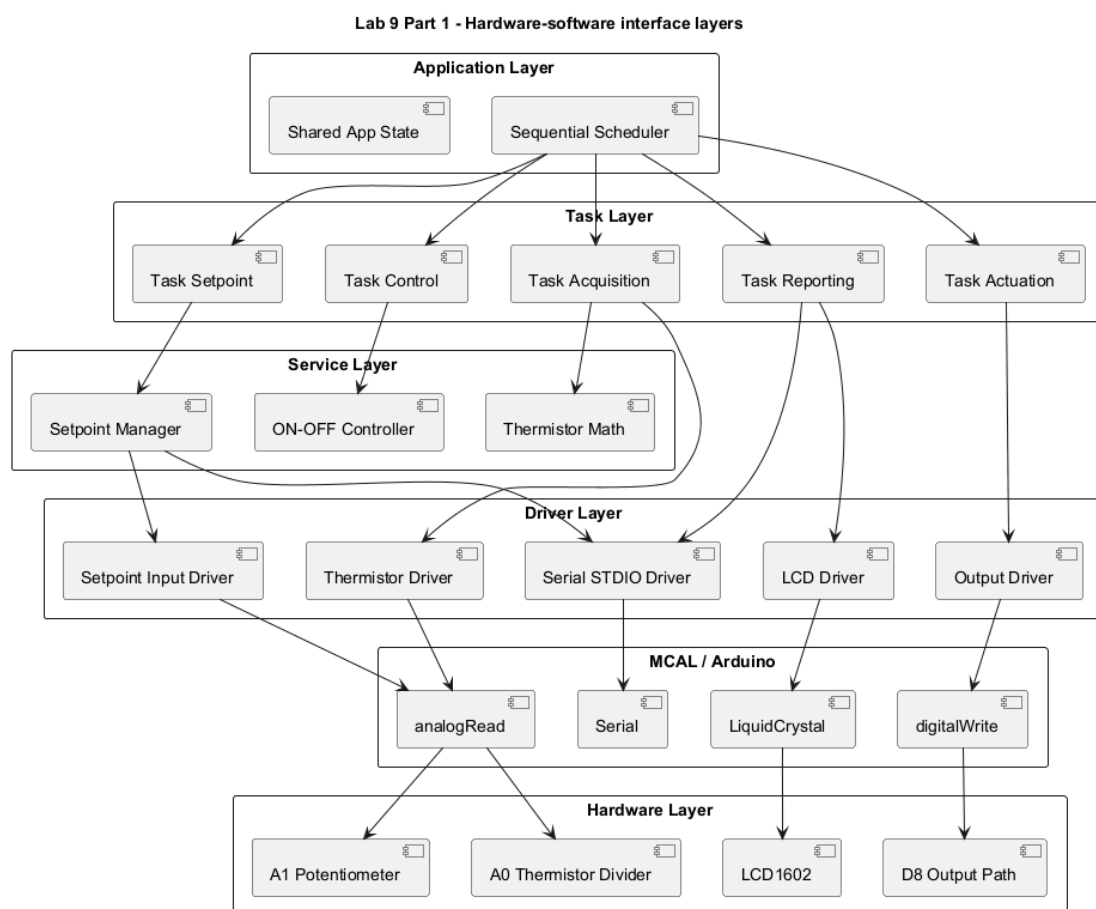


Figure 2: Hardware-software interface and layered architecture

The architectural diagrams satisfy the guideline requirement to show both the com-

plete system structure and the hardware-software interface.

2.5 Behavioral modeling

The application executes periodic sequential tasks. At runtime it acquires the analog sensor signal, converts it into temperature, selects the active setpoint source, computes the hysteresis thresholds, evaluates the acceptable operating range, updates the binary output, and finally reports the current state to LCD and serial interfaces.

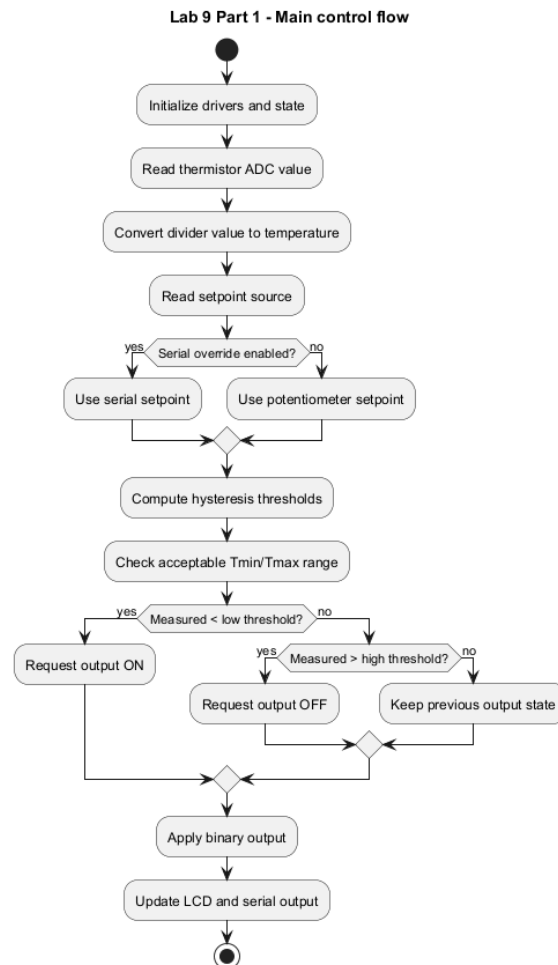


Figure 3: Main control flow for ON-OFF control with hysteresis

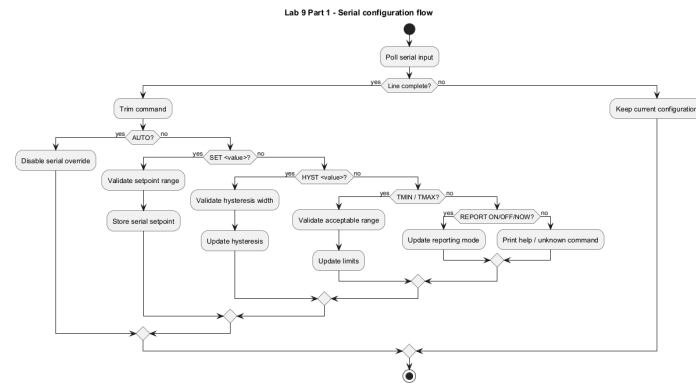


Figure 4: Serial configuration and setpoint management flow

Lab 9 Part 1 - Binary output state machine with hysteresis

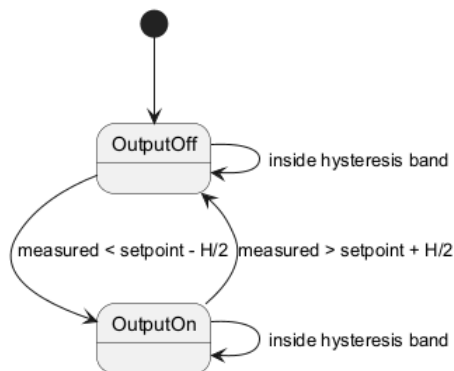


Figure 5: Binary actuator state machine with hysteresis transitions

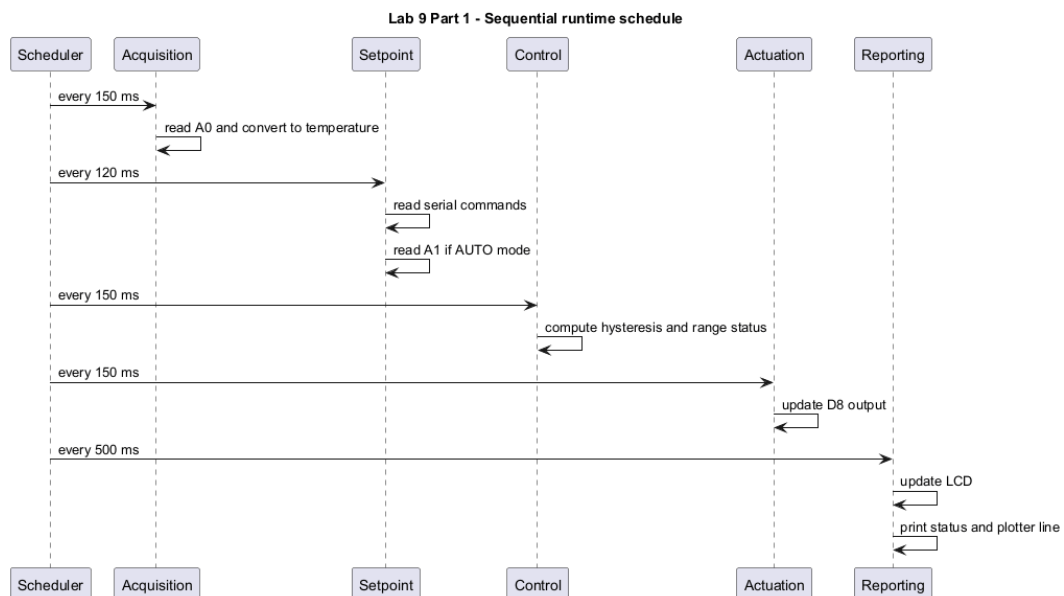


Figure 6: Sequential runtime sequence of the scheduled tasks

These diagrams cover the structural, behavioral, and interface views requested by the project documentation.

2.6 Test scenarios and validation criteria

ID	Scenario	Req.	Pass criterion
T1	Change the simulated thermistor input and observe the measured value.	R1	measured value changes in reports
T2	Rotate the setpoint potentiometer in automatic mode.	R2	setpoint follows hardware input
T3	Send SET, UP, DOWN, and AUTO commands.	R2, R6	setpoint and control mode update correctly
T4	Send HYST, TMIN, and TMAX commands.	R3, R8	runtime parameters update correctly
T5	Set measured value below the lower threshold.	R3, R4	output turns ON
T6	Set measured value above the upper threshold.	R3, R4	output turns OFF
T7	Keep measured value inside the hysteresis band after a transition.	R3, R12	output keeps previous state
T8	Observe LCD content during runtime.	R5	LCD shows value, setpoint, state, and mode
T9	Observe terminal status messages and plotter output.	R6, R7	report is readable and plotter data is generated
T10	Inspect project structure and modules.	R9	files are separated by responsibility
T11	Review periodic scheduler implementation.	R10	sequential tasks and recurrences are explicit
T12	Run <code>pio run</code> .	R11	build succeeds
T13	Review final source tree.	R13	only Part 1 logic is implemented

Table 3: Validation scenarios for Lab 9 Part 1

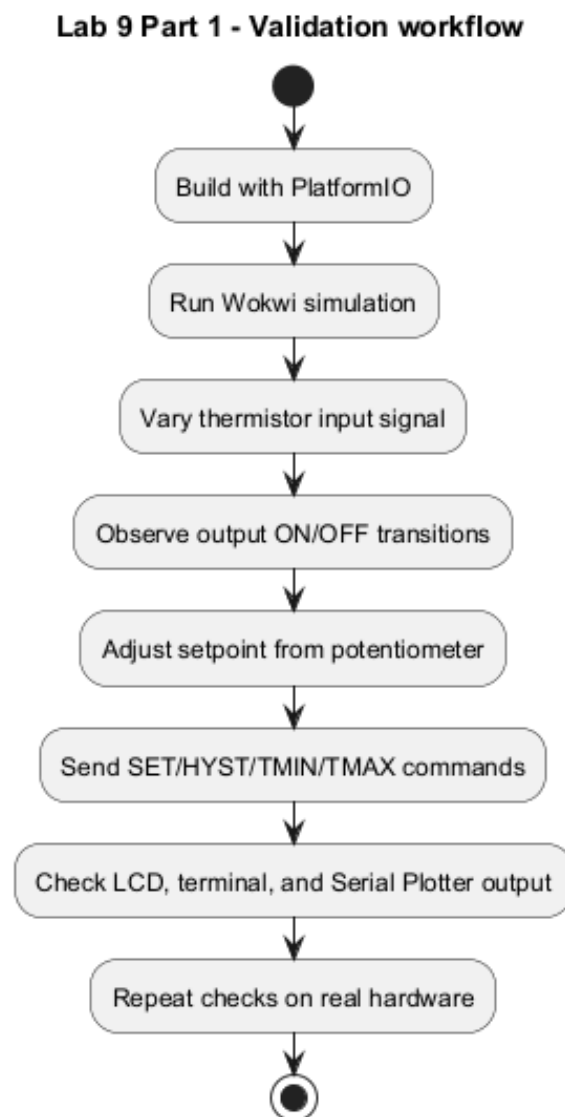


Figure 7: Validation workflow used for the application

Section conclusion

The design phase produced the requirements baseline, structural and behavioral models, and validation criteria needed to implement and verify the final laboratory application in a methodical way.

3. Hardware and software implementation

3.1 Electrical configuration

The final electrical configuration used for implementation and hardware testing includes:

- thermistor divider on A0, with the thermistor connected to 5V and the 10k fixed resistor connected to GND;

- setpoint potentiometer on A1;
- binary heating/output path on D8;
- LCD1602 in 4-bit mode using D7, D6, D5, D4, D3, and D13;
- serial USB interface at 115200 baud.

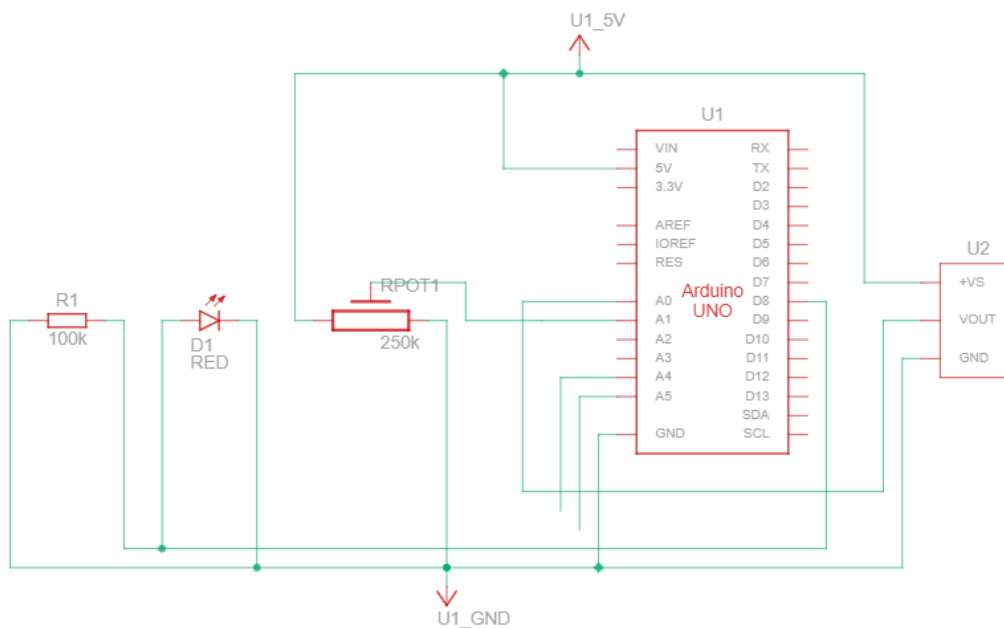


Figure 8: Electrical schematic used for the final Lab 9 implementation

3.2 Source code organization

The project is organized modularly in Lab9/Code:

- Lab9/Code/src/main.cpp: program entry point.
- Lab9/Code/src/app/app_controller.cpp: initialization and sequential scheduler.
- Lab9/Code/src/app/app_state.cpp: shared runtime state.
- Lab9/Code/src/tasks/task_acquisition.cpp: analog acquisition and temperature update.
- Lab9/Code/src/tasks/task_setpoint.cpp: local and serial setpoint update.
- Lab9/Code/src/tasks/task_control.cpp: ON-OFF decision and range evaluation.
- Lab9/Code/src/tasks/task_actuation.cpp: binary output application.
- Lab9/Code/src/tasks/task_reporting.cpp: LCD, serial, and plotter reporting.

- Lab9/Code/src/services/thermistor_math.cpp: divider and thermistor conversion formulas.
- Lab9/Code/src/services/on_off_controller.cpp: hysteresis control logic.
- Lab9/Code/src/services/setpoint_manager.cpp: serial interpreter and runtime configuration.
- Lab9/Code/src/drivers/*.cpp: hardware-facing modules for input, output, LCD, and STDIO.

3.3 Critical implementation sections

Sequential scheduler

```
1 ScheduledTask g_tasks[] = {  
2     {taskAcquisitionRun, kAcquisitionPeriodMs, OUL},  
3     {taskSetpointRun, kSetpointPeriodMs, OUL},  
4     {taskControlRun, kControlPeriodMs, OUL},  
5     {taskActuationRun, kActuationPeriodMs, OUL},  
6     {taskReportingRun, kReportingPeriodMs, OUL},  
7 };
```

Hysteresis controller

```
1 if (measuredTemperatureC < lowThreshold) {  
2     return ActuatorState::On;  
3 }  
4  
5 if (measuredTemperatureC > highThreshold) {  
6     return ActuatorState::Off;  
7 }  
8  
9 return previousState;
```

Thermistor conversion

```
1 return kSeriesResistorOhms *  
2     ((static_cast<float>(kAdcMaxValue) / static_cast<float>(adcValue)) - 1.0F  
3     );
```

Runtime configuration commands

```
1 SET <tempC> UP / DOWN AUTO  
2 HYST <value> TMIN <value> TMAX <value>  
3 REPORT ON REPORT OFF REPORT NOW  
4 HELP
```

These excerpts illustrate the most important implementation areas: scheduling, hysteresis logic, thermistor signal conversion, and the command interface used for runtime tuning and bonus functionality.

3.4 3.4 Implementation reasoning

The final application uses sequential periodic tasks instead of FreeRTOS because the condition explicitly allows either FreeRTOS or sequential tasks with justified recurrences. This approach keeps the software simple and stable on Arduino Uno while still preserving a modular task decomposition.

The final hardware uses a thermistor divider and a binary output proxy because these components were available for the practical setup. Even though the laboratory examples mention digital sensors and relay actuation for some variants, the current implementation still demonstrates the required Part 1 control loop and documents the adopted interpretation clearly.

Section conclusion

The implementation phase produced a modular, working Part 1 control application aligned with the design artifacts and documented through both software structure and hardware interface descriptions.

4 4. Testing and validation

4.1 4.1 Validation methodology

Validation combines compilation, runtime observation, and hardware checks:

1. build the project with `pio run`;
2. vary the measured temperature on the real setup and observe hysteresis transitions;
3. change the setpoint from potentiometer and serial commands;
4. verify runtime configuration of hysteresis and acceptable limits;
5. inspect LCD output, serial STDIO output, and Serial Plotter data;
6. verify the real hardware wiring for the thermistor divider, LCD, and output path.

4.2 4.2 Validation results

ID	Check	Status	Observation
T1	Measured-value acquisition	Pass	analog feedback is converted into temperature reports
T2	Hardware setpoint control	Pass	potentiometer updates setpoint in automatic mode
T3	Serial setpoint commands	Pass	SET/UP/DOWN/AUTO work as expected
T4	Hysteresis and range commands	Pass	HYST/TMIN/TMAX update the control configuration
T5	Lower-threshold activation	Pass	binary output turns ON below the lower threshold
T6	Upper-threshold deactivation	Pass	binary output turns OFF above the upper threshold
T7	Hysteresis retention	Pass	previous state is preserved inside the hysteresis band
T8	LCD reporting	Pass	measured value, setpoint, source, and output are displayed
T9	Serial and plotter reporting	Pass	status messages and plotter fields are available
T10	Modular code review	Pass	logic is separated by responsibilities
T11	Sequential scheduling	Pass	periodic task recurrences are explicit in the scheduler
T12	Build verification	Pass	PlatformIO build succeeds
T13	Scope review	Pass	no PID or Part 2 logic is present

Table 4: Validation summary for Lab 9 Part 1

4.3 4.3 Proof of operation

The documentation requires screenshots and circuit evidence. The collected proof of operation includes:

- `terminal.png`: serial terminal runtime log;
- `hardware.jpg`: real hardware assembly photo;
- `electrical_schema.png`: exported or redrawn electrical schematic.

```

Commands:
SET <temp> - set serial setpoint in range 18.0..45.0
UP         - increase serial setpoint by 0.5 C
DOWN       - decrease serial setpoint by 0.5 C
AUTO       - use potentiometer as setpoint source
MODE P1    - Part 1 heater ON/OFF mode
MODE P2    - Part 2 PID fan mode
MODE HYBRID - heater ON/OFF + PID fan
HYST <value> - set hysteresis width in range 0.2..10.0 C
TMIN <value> - set acceptable minimum temperature
TMAX <value> - set acceptable maximum temperature
KP <value>   - set PID proportional gain
KI <value>   - set PID integral gain
KD <value>   - set PID derivative gain
REPORT ON   - enable periodic reporting
REPORT OFF  - disable periodic reporting
REPORT NOW  - print one report immediately
HELP        - print this help

Plotter fields: SetPoint Value Output
Mode=HYBRID Input=POT Source=pot SetPoint=20.24C Value=25.22C Hyst=1.40C HLow=19.54C HHigh=20.94C Tmin=15.00C Tmax=50.00C Range=OK Heater=OFF Fan=100%
Kp=35.00 Ki=0.80 Kd=12.00
SetPoint:20.24 Value:25.22 Output:-100
Mode=HYBRID Input=POT Source=pot SetPoint=20.24C Value=25.22C Hyst=1.40C HLow=19.54C HHigh=20.94C Tmin=15.00C Tmax=50.00C Range=OK Heater=OFF Fan=100%
Kp=35.00 Ki=0.80 Kd=12.00
SetPoint:20.24 Value:25.22 Output:-100

```

Figure 9: Serial terminal output during ON-OFF control operation

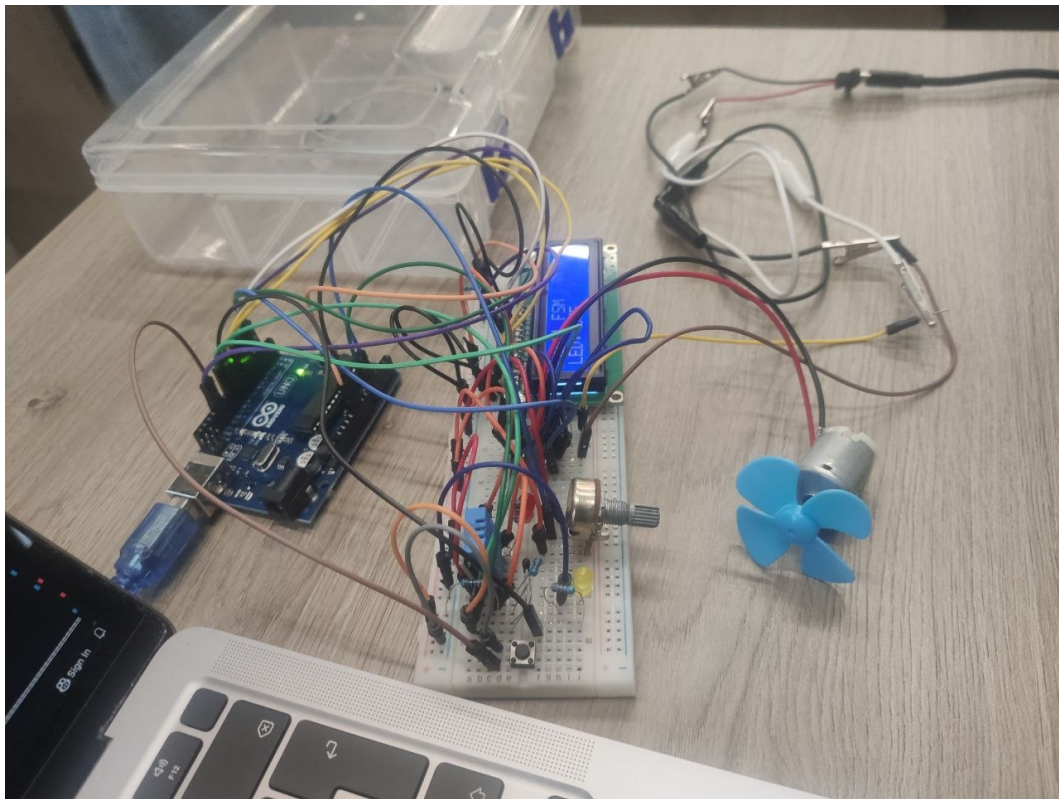


Figure 10: Real hardware assembly used for thermistor-based control

4.4 Hardware observations

Real hardware testing confirms the thermistor behavior, LCD connectivity, and binary output path. The experimental observations show that the controller responds correctly to setpoint, hysteresis, and acceptable range modifications, while the physical placement of the heating resistor relative to the thermistor strongly influences the measured temperature dynamics.

Section conclusion

Testing and validation confirm that the final application satisfies the functional and structural goals of Lab 9 Part 1, while also documenting the limitations and practical lessons observed during real hardware evaluation.

General conclusions

The final Lab 9 project is a complete Part 1 implementation of automatic ON-OFF control with hysteresis for Arduino Uno. It acquires temperature from a thermistor divider, manages a configurable setpoint, hysteresis width, and acceptable operating range, drives a binary output for a heating proxy, and reports all relevant values through LCD, serial STDIO, and Arduino Serial Plotter.

The project follows the documentation structure required by the course, includes architecture and behavioral diagrams, provides explicit validation criteria, and keeps the solution limited to Part 1. The final system also highlights practical lessons regarding thermistor conversion, output polarity, LCD interfacing, and the thermal coupling between heater and sensor.

Possible improvements include using a better thermal actuator driver stage, adding persistent storage for configuration values, introducing filtered acquisition for smoother control, and extending the system toward PID control in a future Part 2 implementation.

AI tool usage note

During the preparation of this report, the author used OpenAI GPT-based tooling for content generation and restructuring. The resulting information was reviewed, validated, and adjusted according to the project requirements before final inclusion.

Bibliography

1. Embedded Systems Practical Guide, Technical University of Moldova, 2026.
2. Laboratory specification for automatic control in embedded systems, Part 1 ON-OFF with hysteresis.
3. PlatformIO Documentation, <https://docs.platformio.org/>.
4. Arduino Language Reference, <https://www.arduino.cc/reference/en/>.
5. LiquidCrystal Library Documentation, <https://github.com/arduino-libraries/LiquidCrystal>.
6. PlantUML Language Reference, <https://plantuml.com/>.
7. NTC Thermistor Theory and Beta Model Reference Materials.

Appendix

A. Project artifacts

The implementation is stored in `Lab9/Code/`.

B. GitHub repository

Repository URL: `https://github.com/Tirppy/si-course-repo/tree/main/Lab9/Code`

C. Diagram sources

PlantUML source files are stored in `Lab9/Report/diagrams/`, and rendered images are stored in `Lab9/Report/images/`.

D. Build and run commands

- Build: `pio run` from `Lab9/Code`
- Serial monitor: `pio device monitor -b 115200`